



DEPARTMENT OF MECHANICAL AND INDUSTRIAL
ENGINEERING

TMM4935 - INDUSTRIAL ICT, MASTER'S THESIS

Development of video streaming tool for Remote Train Operations

Author:

Einar Thingstad Myhrvold

Student number:

543951

Supervisor:

Nils O.E. Olsson

Date: 18.03.2026

Preface

This master thesis

The development of ATO at NTNU has been in progress for multiple years and is now in the remote operation phase. Master students at the Department of Mechanical and Industrial Engineering has steadily worked on this project. In the fall of 2024 and spring of 2025 it was from Emilia Kozarevic, and I will continue her and NTNU's research on the topic.

Subject & knowledge listed below has been used.

- TBA4225 - Railway Engineering 2
- IT2810 - Web Development
- TDT4240 - Software Architecture
- TDT4120 - Algorithms and Data Structures

I would like to end the preface by thanking Nils Olsson for the work of supervising me during this semester and the effort he has put in to help me write this paper. I also would like to thank Patrick Urassa (NTNU) and Tomas Rosberg from VTI for discussing and the insight in the project.

Abstract

Norwegian

Table of Contents

List of Figures	v
List of Tables	v
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem	1
1.3 Objectives	2
1.4 Research Questions	2
1.5 Scope and Limitations	2
1.6 Outline of Structure	3
2 Theory	4
2.1 Remote Train Operations	4
2.1.1 Grades of Automation	4
2.1.2 Role of Video in Remote Operation	4
2.1.3 Latency Requirements and Regulations	5
2.2 Video Streaming	5
2.3 Video Codecs	6
2.3.1 H.264	6
2.3.2 H.265	6
2.3.3 MJPEG	6
2.4 Transport Protocols	8
2.4.1 RTP and SRTP	8
2.4.2 UDP and TCP	8
2.5 Streaming Parameters	8
2.5.1 Bandwidth	8
2.5.2 Resolution	8
2.6 Latency Sources in the Streaming Pipeline	8
2.6.1 Capturing	8
2.6.2 Encoding	8
2.6.3 Transmitting	8
2.6.4 Decoding	8
2.6.5 Rendering	8

2.7	GStreamer	8
2.7.1	Pipeline and Elements	8
2.7.2	Pads	8
2.7.3	Sinks	8
2.7.4	Probes	8
2.8	Network and Communication	8
2.8.1	Wi-Fi	8
2.8.2	Cellular and 5G	8
2.8.3	Tailscale	8
2.9	Performance Metrics	8
2.9.1	Latency	8
2.9.2	Packet Loss	8
2.9.3	Throughput	8
2.9.4	CPU Usage	8
2.9.5	Visual Quality	8
2.10	GStreamer	8
3	System Design	10
3.1	Requirements	10
3.2	Architecture	10
3.3	Pipeline Design	12
3.4	Timestamps and Latency Measurement Design	12
3.5	Wi-Fi Network Setup	12
4	Methodology	13
4.1	Literature and Knowledge acquisition	13
4.1.1	Search Strategy and Databases	13
4.1.2	Inclusion Criteria	13
4.1.3	Search Queries and Refinement	14
4.2	Validity	14
4.3	Reliability	14
4.4	Research Design	14
4.5	Development	15
4.5.1	GStreamer Pipeline Development	15
4.6	Experimental Methodology	15
4.6.1	Latency Measurement	15

5	Results	16
6	Conclusion	17
6.1	Evaluation of latency	17
	References	18

List of Figures

1	Physical Setup of Project	10
2	Application and Transport layer Setup of Project	11
3	Sequence Diagram between Sender and Camera before active Stream	12

List of Tables

1	Grades of Automation as defined in IEC 62290–1[3, 4].	4
2	Setup for IP and Port Connection	11
3	Inclusion Criteria for Literature Review	14

1 Introduction

As trains become automated, the need for operators to monitor and control them remotely is growing. To do this safely, the operator needs a live video feed from the train that is stable and fast enough to react to what is happening on the track. This thesis looks at how such a video streaming system can be built, tested and evaluated, with a focus on finding configurations that work well for remote train operation.

1.1 Background and Motivation

As the railway sector transitions toward higher Grades of Automation (GoA), particularly GoA 3 and GoA 4, remote driving becomes an increasingly critical operational capability. Remote driving is not only a natural consequence of reduced onboard personnel at higher GoA levels, but serves as a mandatory safety layer within the ATO architecture enabling operators to intervene when automated systems encounter situations they cannot resolve independently, such as unexpected track conditions or degraded operational modes [source].

A fundamental requirement for safe remote driving is the availability of reliable, low-latency video streaming. When the operator is physically separated from the train, the video feed becomes the primary source of situational awareness, and its quality and responsiveness directly determine whether timely and safe intervention is possible [source]. Industry experience further demonstrates that remote operation can support a range of practical tasks including shunting, depot movements, and vehicle preparation provided that the communication link meets sufficient performance criteria [source].

Understanding the relationship between latency and human perception is equally important in this context. Research on human factors in remote operation indicates that video delay, bitrate, and stream stability all influence an operator's ability to detect obstacles, interpret dynamic scenes, and execute corrective actions under time pressure [source]. Identifying the latency boundaries within which safe remote operation remains feasible is therefore a key step toward enabling higher automation levels in railway systems.

This thesis is developed within an ongoing research collaboration at NTNU, in partnership with VTI and other European railway organizations, contributing to the technical groundwork required to make remote train driving a safe and practical reality in future railway deployments.

1.2 Problem

The current remote train operation setup relies on a commercial product, Voysys, which imposes significant limitations on the project's ability to test and develop the system further. As a closed external solution, it restricts access to the underlying pipeline, making it difficult to adjust configurations, measure internal latency, or experiment with different streaming approaches. In a research context where iterative testing and configuration control are essential, this represents a fundamental obstacle. In addition, licensing such a product over the course of a long term research project represents a considerable and ongoing cost.

At the same time, there is limited existing research that addresses video streaming specifically in the context of remote train operation. Most work on low latency video transmission is either too general or focused on other domains, leaving a gap when it comes to understanding what configurations and compromises are most suitable for railway use cases [source].

Building a custom streaming solution also comes with its own set of challenges. Network conditions such as signal strength, interference, and varying load introduce unpredictable variability in transmission quality. Choosing the right codec, transport protocol, and encoder settings requires systematic testing, as each parameter affects latency, quality, and resource usage in ways that are not always predictable. Balancing video quality against latency is a difficult comparison and testing

these differences in settings on real hardware in a realistic setup adds further complexity. Without a flexible and open pipeline, it is difficult to isolate and measure where latency is introduced and how different configurations affect overall system performance.

1.3 Objectives

The main objective of this thesis is to design and implement a low-latency video streaming pipeline tailored specifically for remote train operation, serving as a practical replacement for the current commercial solution. The pipeline should be functional, flexible, and open enough to support continued development and testing within the project. More specifically, the objectives are to:

1. O1: Develop a working GStreamer-based streaming pipeline that meets the low-latency requirements of remote train operation
2. O2: Implement a latency measurement framework that allows end-to-end delay to be captured, logged, and compared across different configurations
3. O3: Evaluate the pipeline across multiple configurations, including different codecs, transport protocols and network settings, to identify the interplay between latency and video quality
4. O4: Design the system to be dynamic and extensible, allowing additional cameras and new configurations to be added with minimal effort
5. O5: Deliver a stable and usable prototype that is good enough for practical use during the current development period and serves as a foundation for future work

The result should be a system that gives the project full control over the streaming pipeline, removes dependency on external licensed software, and produces reliable data to guide future configuration decisions.

1.4 Research Questions

The following research questions guide the experimental work of this thesis:

- RQ1: What end-to-end latency and video quality can be achieved in a GStreamer-based video streaming pipeline for remote train operation, and which codec and transport configuration performs best?
- RQ2: How do network conditions, including transport protocol and available bandwidth, affect streaming latency and reliability in a remote train operation context?
- RQ3: What are the practical interplay between latency, video quality, and resource usage across different streaming configurations, and which configuration is most suitable for remote train operation?

1.5 Scope and Limitations

The scope of this thesis covers the design, implementation and experimental evaluation of a custom video streaming pipeline for remote train operation. The system is developed as a proof of concept and is not intended as a deployment ready solution. Development and pre testing was conducted in a controlled indoor environment, while final testing was performed from a moving vehicle to simulate realistic train movement conditions.

The experimental evaluation focuses on the parameters identified as most relevant to remote train operation based on prior research [source: prosjektoppgave], and selected based on time and hardware availability. The configurations tested include codec selection (H.264, H.265 and MJPEG),

transport protocol (UDP and TCP), encryption overhead (SRTP), bandwidth conditions, resolution, and GPU versus CPU based encoding. While GStreamer allows a large number of additional parameters to be adjusted, such as buffer sizes, queue configurations, bitrate control modes and keyframe intervals, expanding the test matrix further was not feasible within the time constraints of a single master thesis. The chosen parameters are considered sufficient to identify the most significant performance differences relevant to the use case.

Latency is measured end to end between the point where an RTP packet enters the sender pipeline and the point where the corresponding decoded frame arrives at the receiver display sink. Camera capture latency and screen rendering latency are not included in these measurements, as they occur outside the instrumented portion of the pipeline and are considered largely constant across configurations. Clock synchronization between the sender and receiver relies on internet based time sources, introducing a small margin of uncertainty estimated at a few milliseconds, which is acknowledged when interpreting absolute latency values.

The thesis does not include testing on an actual operational train. A moving vehicle was used to simulate train conditions, but real world railway infrastructure, operational speeds, and trackside network coverage are outside the scope of this work and represent a natural next step for future validation.

The thesis does not include formal safety certification or a RAMS analysis in accordance with EN 50126. While the system is developed with safety critical use in mind, evaluating compliance with railway safety standards is considered outside the scope of this work and is left for future development phases.

1.6 Outline of Structure

The remainder of this thesis is organized as follows:

Chapter 2 presents the theoretical background underlying the work. This includes an overview of Automatic Train Operation and the role of remote driving at higher GoA levels, fundamentals of video streaming including codecs, transport protocols and payloading, an introduction to GStreamer and its core concepts, the main sources of latency in a streaming pipeline, relevant aspects of cellular and WiFi communication, and the metrics used to evaluate streaming performance.

Chapter 3 describes the design of the streaming system. This covers the requirements that shaped the system, the overall architecture and component choices, the GStreamer pipeline design for both sender and receiver, the approach to timestamp based latency measurement, and the network setup used in testing.

Chapter 4 explains how the research was conducted. This includes the overall research design, the iterative development process and how it shaped the system over time, the GStreamer pipeline development process, and the experimental and latency measurement methodology used to evaluate the system.

Chapter 5 presents the experimental results. This covers a comparison of GStreamer configurations across different codecs, transport protocols and encoding approaches, latency measurements per configuration, the effect of network conditions on streaming performance, and the interplay between video quality and latency across configurations.

Chapter 6 interprets and reflects on the results in relation to the research questions and the broader context of remote train operation.

Chapter 7 summarizes the key findings and contributions of the thesis and outlines directions for future work, including testing on real railway infrastructure and further expansion of the configuration space.

2 Theory

This chapter presents the theoretical background needed to understand the work carried out in this thesis. It covers the principles of Automatic Train Operation and the role of remote driving within that framework, the fundamentals of video streaming including codecs and transport protocols, an introduction to GStreamer as the core technology used in the pipeline, the main sources of latency in a streaming system, and relevant characteristics of the communication technologies used in testing.

2.1 Remote Train Operations

Remote Train Operations (RTO) refers to the ability to monitor and control a train from a location outside the cab, typically a centralised control room, using live video feeds and remote control interfaces. As railways move toward higher levels of automation, RTO has become an important part of safe and practical deployment. It allows human operators to intervene when automated systems encounter situations they cannot handle on their own, and it serves as a required fallback layer for trains operating without a driver or on-board staff[1, 2].

2.1.1 Grades of Automation

The Grade of Automation (GoA) is a classification system defined in the international standard IEC 62290–1 that describes how much of the train’s operation is handled automatically versus by a human[3]. The standard defines five levels, from GoA 0 to GoA 4, where each higher level shifts more responsibility from the driver to the automated system. Table 1 summarises the levels.

Table 1: Grades of Automation as defined in IEC 62290–1[3, 4].

GoA	Name	Description
GoA 0	On-sight operation	Fully manual with no automatic protection.
GoA 1	Manual operation	Driver controls the train; Automatic Train Protection (ATP) enforces speed limits and safe separation.
GoA 2	Semi-automated (STO)	ATO handles acceleration and braking; a driver remains on board for doors, obstacle response, and degraded modes.
GoA 3	Driverless (DTO)	No driver needed in normal operation. On-board staff may be present for passenger assistance and emergencies.
GoA 4	Unattended (UTO)	Fully automatic operation with no staff on board. Remote supervision and control are required for incidents.

RTO is most relevant from GoA 3 upward. At GoA 3, on-board staff can still respond to incidents directly, but remote oversight is used for operational management. At GoA 4, where no staff are on board, RTO becomes the primary mechanism for human intervention when automated systems fail or encounter unexpected conditions[5, 6].

2.1.2 Role of Video in Remote Operation

A remote operator who cannot physically see or hear the train must rely entirely on what the system provides. In practice, this means live video from cameras mounted on or near the train is the most important source of situational awareness[7, 8]. Without a stable and clear video feed, the

operator cannot assess track conditions, detect obstacles, interpret signals, or make safe decisions about speed and braking.

Research on human factors in remote driving consistently shows that video quality has a direct effect on operator performance. Studies on bitrate, frame rate, and resolution indicate that reductions in bitrate lead to measurable drops in both reaction speed and accuracy, while changes in frame rate have a smaller effect within typical operating ranges[7]. This suggests that encoding and transmission choices in the video pipeline are not just technical decisions but directly influence operational safety.

Beyond individual video quality, remote train driving typically requires multiple simultaneous camera views. A camera facing forward is needed for obstacle and signal detection, while side and rear cameras support door operations, platform supervision, and shunting tasks[1, 2]. The system developed in this thesis streams three concurrent camera feeds, reflecting this multi-view requirement.

2.1.3 Latency Requirements and Regulations

Latency is the central performance parameter in any RTO video system. If the delay between what happens on the track and what the operator sees on screen is too large, safe intervention becomes impossible. The challenge is that no single, universally agreed threshold exists for railway remote operation, and most values currently used in the field are borrowed from research on cars, drones, and cranes[9].

The most relevant regulatory reference comes from 3GPP TS 22.289, a technical specification developed in cooperation with the International Union of Railways (UIC) and the European Telecommunications Standards Institute (ETSI)[10]. It classifies remote video control at GoA 3 and GoA 4 as *Very Critical Video Communication* and sets a target end-to-end latency of ≤ 100 ms at speeds up to 500 km/h, or ≤ 10 ms at speeds up to 40 km/h, with a reliability requirement of 99.9% and a data rate of 10–20 Mbps. These values are defined at the communication interface and are not yet formally adopted into ERA’s Technical Specifications for Interoperability (TSIs)[10].

Empirical studies from railway demonstrations report glass-to-glass (G2G) latency values of 340–380 ms for tramway remote driving using commercial systems[11], while the Kozarevic study at NTNU measured an average of around 212 ms using a GPS-synchronised measurement method[8]. Human factors research from other vehicle domains suggests that operators begin to adjust their behaviour at delays above 250–300 ms, and that latency variability can be more disruptive than a consistently higher but stable delay[9, 12].

These findings together motivate the work in this thesis. To build and evaluate a custom streaming pipeline where each component of the latency budget can be measured, understood, and optimised, rather than relying on a black-box commercial system.

2.2 Video Streaming

Video streaming is the continuous transmission of video data from a source to a receiver over a network, allowing the receiver to display the content in real time without storing the full recording first[13]. In a remote train operation context, this means live footage from cameras on or near the train is captured, compressed, sent across a network, and displayed on the operator’s screen with as little delay as possible.

A video streaming system can be thought of as a chain of components, each with its own role. At the sending end, raw video is captured by a camera and handed to an encoder, which compresses it into a format suitable for transmission. The compressed data is then packaged into a transport protocol and sent across the network. At the receiving end, the process is reversed: packets are reassembled, decoded back into raw frames, and rendered on screen. The choices made at each step, which codec, which protocol, which network directly affect the end-to-end latency and video quality that the remote operator experiences[9, 14].

2.3 Video Codecs

2.3.1 H.264

H.264, also known as Advanced Video Coding (AVC), is a video compression standard developed jointly by the ITU-T Video Coding Experts Group and the ISO/IEC Moving Picture Experts Group[15]. It has become the dominant codec for real-time video applications including video conferencing, IP surveillance, and live streaming, due to its balance between compression efficiency, computational cost, and broad hardware support[16].

The codec reduces redundancy across two dimensions. Within a single frame, intra-frame prediction estimates pixel values from neighbouring blocks and stores only the difference. Across consecutive frames, inter-frame prediction identifies how blocks have moved and stores motion vectors rather than full pixel data, allowing H.264 to represent video at significantly lower bitrates than older standards such as MPEG-2 while maintaining comparable visual quality[15].

A key concept is the Group of Pictures (GOP), which defines how often a full intra-coded frame, called an I-frame, appears in the stream. I-frames can be decoded independently without reference to other frames, making them essential for stream recovery after packet loss. For real-time applications, short GOP intervals are preferred to reduce both start-up delay and recovery time, at the cost of a slightly higher average bitrate[16].

2.3.2 H.265

H.265, also known as High Efficiency Video Coding (HEVC) and formally standardised as ITU-T H.265 and ISO/IEC 23008-2, is the successor to H.264 developed jointly by the same two standardisation bodies and published in 2013[17]. Its primary goal was to achieve roughly 50% better compression compared to H.264 at equivalent visual quality, making it particularly attractive for high resolution content and bandwidth constrained networks[18].

HEVC achieves its efficiency gains by replacing the fixed 16×16 macroblock structure of H.264 with a flexible Coding Tree Unit (CTU) architecture that supports block sizes up to 64×64 pixels. Larger blocks handle uniform image regions efficiently, while smaller blocks are used in areas with fine detail or motion. This adaptive structure allows the encoder to allocate coding effort where it is most needed, reducing the total number of bits required to represent a frame at a given quality level[17].

The main difference compared to H.264 is computational cost. Encoding with H.265 can require significantly more processing power, which increases encoding latency and places greater demand on the sender hardware[18]. In a real-time remote operation context this is a relevant concern, as any added encoding delay directly contributes to the total end-to-end latency experienced by the operator. For this reason H.265 is included in this thesis as a comparison codec, to evaluate whether its bandwidth savings justify the additional encoding overhead under the test conditions used.

2.3.3 MJPEG

Motion JPEG (MJPEG) is a video format in which each frame is compressed independently as a JPEG image, with no reference to any other frame in the sequence[19]. This purely intraframe approach contrasts with H.264 and H.265, which both rely on temporal prediction across frames to achieve compression.

Because each frame carries all the data needed for its own decoding, MJPEG has two notable properties relevant to remote operation. First, packet loss affects only the frame it occurs in, with no propagation of errors to subsequent frames. Second, encoding and decoding complexity is low and predictable, as the encoder processes each frame in isolation without any lookahead or motion estimation[20]. These properties can result in lower and more consistent encoding latency

compared to inter-frame codecs under the same hardware conditions.

The main limitation of MJPEG is its high bandwidth consumption. Without temporal compression, every frame must carry its full spatial data, which results in bitrates several times higher than H.264 at equivalent quality[21]. This makes MJPEG less suitable for constrained or variable networks, where the higher data rate increases the risk of congestion and packet loss. In this thesis MJPEG is tested as a reference point to understand how the absence of inter-frame prediction affects the overall latency and quality measurements of the streaming pipeline.

2.4 Transport Protocols

2.4.1 RTP and SRTP

2.4.2 UDP and TCP

2.5 Streaming Parameters

2.5.1 Bandwidth

2.5.2 Resolution

2.6 Latency Sources in the Streaming Pipeline

2.6.1 Capturing

2.6.2 Encoding

2.6.3 Transmitting

2.6.4 Decoding

2.6.5 Rendering

2.7 GStreamer

2.7.1 Pipeline and Elements

2.7.2 Pads

2.7.3 Sinks

2.7.4 Probes

2.8 Network and Communication

2.8.1 Wi-Fi

2.8.2 Cellular and 5G

2.8.3 Tailscale

2.9 Performance Metrics

2.9.1 Latency

2.9.2 Packet Loss

2.9.3 Throughput

2.9.4 CPU Usage

~~2.9.5 Visual Quality~~

2.10 GStreamer

GStreamer is an open-source multimedia framework that allows developers to build pipelines for processing and transmitting audio and video streams. It is widely used in both research and

industry due to its flexibility, broad codec support, and active community [source]. Unlike closed commercial solutions, GStreamer provides full access to every stage of the streaming process, making it well suited for experimentation and performance tuning.

The core concept in GStreamer is the pipeline, which is a chain of processing elements connected in sequence. Each element performs a specific task, such as capturing video from a camera, encoding the stream, packeting it for network transmission, or rendering it on a display. Data flows through the pipeline from one element to the next, and each element can be configured independently. This modular design makes it straightforward to swap out components, such as replacing one codec with another, without rebuilding the entire pipeline [source].

Pipelines in GStreamer can be constructed and launched directly from the command line using the `gst-launch-1.0` tool, or built programmatically using languages such as Python through the GStreamer Python bindings. For this project, both approaches are used, with Python providing the flexibility to control pipeline behaviour, handle events, and integrate latency measurement logic alongside the streaming process [source].

What makes GStreamer particularly relevant for this thesis is the level of control and transparency it offers. Every element in the pipeline can be monitored and tuned, latency can be measured at specific points, and configurations can be changed and tested systematically. This makes it a practical and powerful foundation for building a low-latency video streaming system for remote train operation.

3 System Design

This chapter describes the design of the video streaming system developed in this thesis. It outlines the requirements that shaped the system, the overall architecture, the pipeline design, the approach used for timestamp-based latency measurement, and the Wi-Fi network setup used during testing. Together these sections document the reasoning behind the key design decisions made throughout the development process.

3.1 Requirements

The requirements for the system are shaped by the operational context of remote train operation, where reliability and low latency are critical. Unlike a controlled lab setting, a moving train introduces real-world challenges such as varying network conditions, physical vibrations, and the need for the system to perform consistently during operation.

The key requirements identified for the system are:

Low latency: The pipeline must deliver video with as little end-to-end delay as possible, as latency directly affects the remote operator’s ability to perceive and react to the environment safely.

Reliability: The system must maintain a stable stream under varying network conditions, as interruptions or degraded quality during operation could have serious safety implications.

Multi-camera support: The system must be capable of handling multiple camera streams simultaneously, reflecting the practical needs of a remote driving setup where several viewing angles are required.

Deployable on a moving train: The system must function in a real train environment, accounting for the network and physical conditions that come with that setting.

Reproducibility: The setup must be straightforward to reproduce and redeploy, both for continued development within the project and for potential use by other researchers or partners in the collaboration.

3.2 Architecture

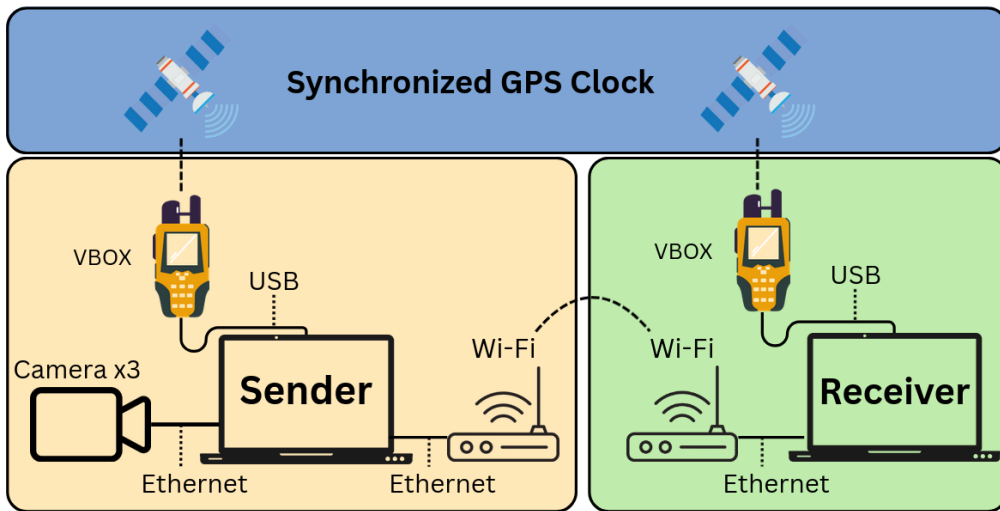


Figure 1: Physical Setup of Project

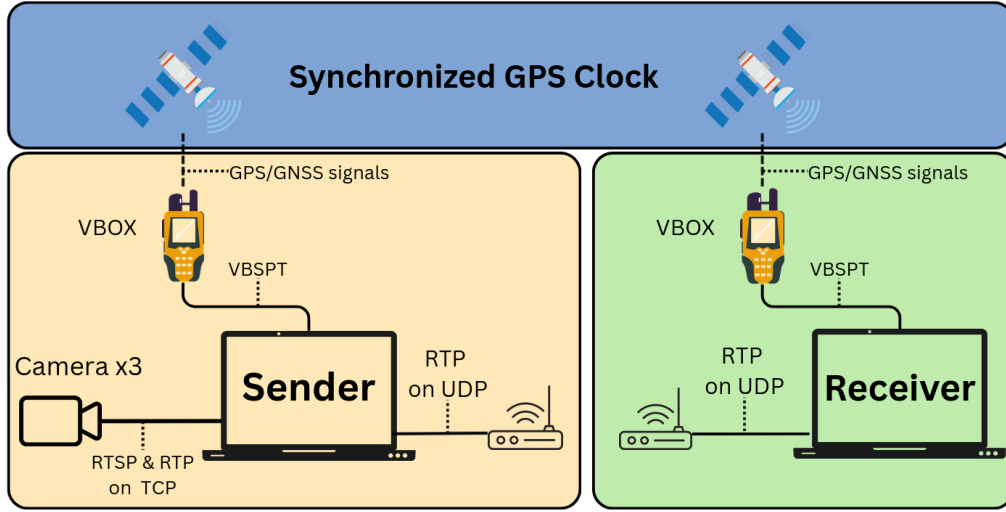


Figure 2: Application and Transport layer Setup of Project

Table 2: Setup for IP and Port Connection

Camera	Camera IP	Connection interface	Local IP	Receiver IP	RTP Port
Camera 1	192.168.0.100	eth0	192.168.0.50	10.238.111.249	5000
Camera 2	192.168.1.101	eth1	192.168.1.50	10.238.111.249	5002
Camera 3	192.168.2.102	enp0s31f6	192.168.2.50	10.238.111.249	5004

IP Cameras: Logged in with SADE, iVMS and Hikvision, to be set to standard IP address of 192.168.1.100, 101 and 102. This will remain IP address until manually overwritten.

Important to separate: Transport Layer: TCP, UDP Application Layer: RTSP, RTP TCP perfect, correct order and intact. UDP speedier but less reliable

Between Cam and Laptop, TCP. A NetCat TCP handshake to port 554 to check its availability.

Sender Computer Sets local IP address to 192.168.1.20 to make sure to be on the same subnet as the IP cameras. Pings local IP address of cameras connected to confirm possible connection. Checks RTSP port is available by using NetCat. NetCat sends a TCP handshake. RTSP message over TCP, asking for stream. Describe Stream, Camera describe, Setup up stream, Camera ready, Play stream. Setup because we ask for protocol TCP, we go Interleaved: RTSP and 2 Channels inside TCP connection. Channel 0 carries RTP (video data) Channel 1 carries RTCP (Control and monitoring QoS, Sync streams, provides feedback)

Real-Time Streaming Protocol control the stream. Using NetCat (needs download) to check if the RTSP port is open

Pulls the stream from the camera over RTSP/TCP, with no jitter buffer “latency = 0”

Camera sends: _____ (perfectly even) Network delivers: _____
 _____ (uneven gaps)

Without buffer: _____ (choppy, uneven) With buffer: waits... then
 _____ (smooth, even)

Because of local wired connection, the jitter is probably very low, and not adding jitter buffer will not affect much other than lowering latency.

rtpH264depay, Strips the stream of RTP giving raw H.264 NAL units

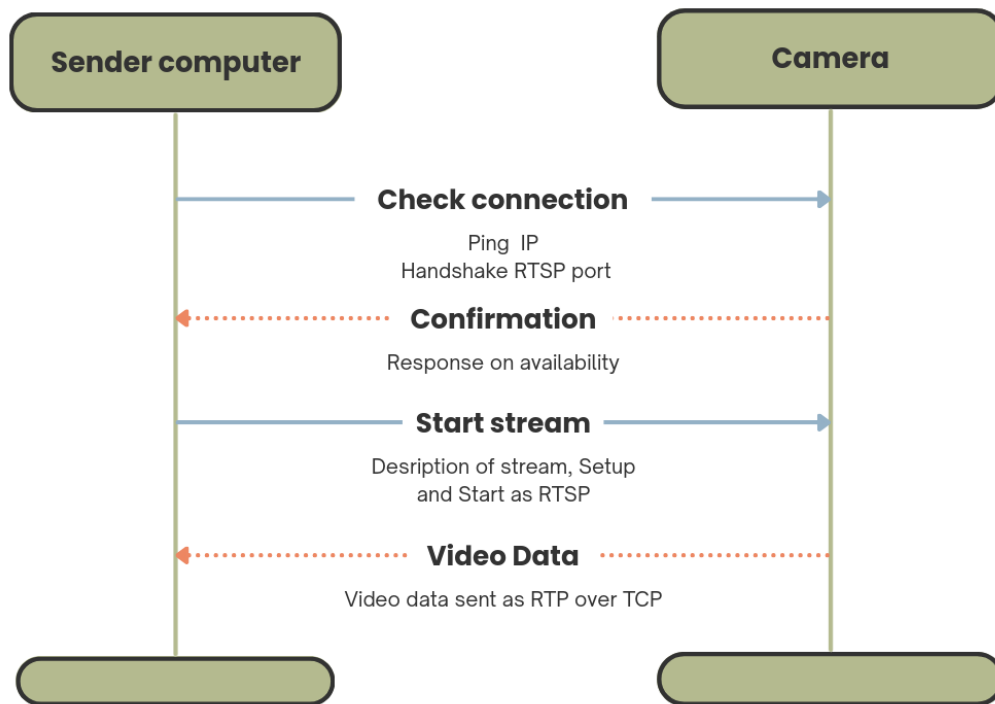


Figure 3: Sequence Diagram between Sender and Camera before active Stream

H.264 video is made up of NAL units (chunks of compressed video). RTP is rules how to split up big NAL units and how to assemble them again. Why Unpack and repack? Control over the stream — you get a clean interception point for your probe config-interval=1 — injecting SPS/PPS every frame so the receiver never gets lost Flexibility — you could insert transcoding, scaling, or other processing between depay and pay if needed later

What is a Probe? In GStreamer, a probe is essentially a spy you attach to a specific point in the pipeline. It lets you intercept data as it flows through, look at it, and optionally modify or drop it — all without interrupting the pipeline itself.

What is a Pad? Before understanding probes you need to understand pads — because probes attach to pads, not to elements directly. Every GStreamer element has pads — they are the connection points between elements:

rtph264pay, Package back into RTP packets, injects SPS/PPS every frame “config-interval-1”
udpsink, sends packets to receiver with UDP

Receiver Computer: Listens to UDP port 5000 rtph264depay: Strips RTP again to get H.264 NAL units
h264parse: Parses and structures the bitstream avdec-h264: Software decodes H.264 to raw frames (this is the expensive step)
autovideosink: Displays on screen (sync=false = no clock sync, lowest latency)

3.3 Pipeline Design

3.4 Timestamps and Latency Measurement Design

3.5 Wi-Fi Network Setup

4 Methodology

4.1 Literature and Knowledge acquisition

The knowledge needed to carry out this thesis was acquired through an iterative process that combined systematic literature review with hands-on practical exploration. This approach, often referred to as an Iterative Literature and Practice-Driven Approach, is characterized by a continuous cycle where theory and practice inform each other rather than following a strict linear sequence [Wohlin et al. (2012)]. Instead of completing all literature review before beginning practical work, the two activities run in parallel, with new practical challenges prompting targeted literature searches and new theoretical insights shaping the direction of experiments. This makes it particularly well suited for applied engineering research where the problem space is not fully known at the outset and understanding develops gradually through doing [Hevner et al. (2004) Saunders et al. (2019)].

The process began by identifying the core knowledge areas required before any development could start. This included understanding the operational context of remote train operation and Automatic Train Operation, the fundamentals of video streaming and relevant transport protocols, and the capabilities and structure of GStreamer as a framework. These areas formed the initial scope of the literature review and gave enough grounding to begin practical work.

Academic literature was gathered primarily through Google Scholar and IEEE Xplore, using search terms related to low-latency video streaming, remote train operation, GStreamer pipeline design, and network performance for real-time applications. Sources were selected based on relevance to the problem, publication quality, and recency where applicable. In addition to academic papers, official technical documentation was used extensively, particularly the GStreamer documentation and protocol specifications for RTP and UDP, which provided the practical detail that academic sources alone could not cover.

Once a sufficient theoretical foundation was in place, practical exploration began through hands-on work with GStreamer. Building and testing pipelines from the command line and through Python quickly revealed challenges that existing literature did not fully address, such as specific configuration trade-offs, unexpected latency behavior, and hardware compatibility issues. Each of these practical obstacles prompted a return to the literature, this time with more precise and targeted questions. New findings were then brought back into the development process, tested, and evaluated against real behaviour.

This cycle of reading, building, observing, and searching again repeated throughout the entire development phase. The result is that the theoretical foundation of this thesis did not precede the practical work, but grew alongside it, shaped by the specific demands of the problem at hand.

4.1.1 Search Strategy and Databases

A multi-platform search approach was utilized to retrieve a wide array of high-quality sources. The primary databases included:

- Scopus: For retrieving peer-reviewed, impactful scientific articles.
- Google Scholar: For broader academic and institutional literature.
- GStreamer docs: For specifics on GStreamer development.

4.1.2 Inclusion Criteria

To ensure the study is based on the relevant and current information, specific criteria were applied to filter the search results. Given the rapid pace of technological change, a focus was placed on

recent publications. Where modern papers have referenced to older papers, I have include some as to say and show where certain numbers are coming from and how they have influenced results.

Table 3: Inclusion Criteria for Literature Review

ID	Inclusion Criteria (IC)
IC1	Publication Date: Between 2020-2026
IC2	Language: Written in English
IC3	Document Type: Primarily "Article", "Conference Paper" or "Thesis"

4.1.3 Search Queries and Refinement

Several digital tools as mentioned below were used for this paper. The main purpose of the usage was to enhance clarity, assisting in latex, and in general increase quality. Specifics include writing reference list in correct format. Structuring sentences and paragraphs. Checking for typos and grammatical errors.

- OpenAI's ChatGPT: Employed as a helpful resource for LaTeX formatting suggestions, generating structured content (tables and lists), translating between Norwegian and English, and reviewing text for synonyms and restructuring ideas to ensure arguments were effectively communicated.
- Google Gemini: Used for similar fields as ChatGPT, but also for general proofreading, and refining sentence structure and tone to maintain a high standard of academic writing.
- Anthropic's Claude.ai for discussion and development of script syntax.
- Visual Studio Code: was used as the main LaTeX editor because of its powerful extensions for LaTeX support, syntax highlighting, and version control integration.

4.2 Validity

4.3 Reliability

4.4 Research Design

This thesis follows an applied research approach, combining Design Science Research (DSR) and experimental research to address both the development and evaluation aspects of the work. This combination is well suited for engineering projects where the goal is to produce a functional artifact while also generating knowledge through systematic testing and measurement.

The DSR component governs the design and development of the video streaming pipeline. Following the principles outlined by Hevner et al. [source], the research contribution is embedded in the artifact itself, meaning the pipeline design, the architectural decisions made, and the reasoning behind them all form part of the research output. The problem is clearly grounded in practice, the solution is built and refined iteratively, and the artifact is evaluated against the requirements defined in the previous chapter.

The experimental research component covers the testing and evaluation phase. Drawing on the framework described by Wohlin et al. [source], the pipeline is tested across a range of configurations in a controlled setting. Variables such as codec choice, payload settings, and network conditions are systematically varied, and end-to-end latency is measured and compared across configurations. This allows conclusions to be drawn about which configurations perform best and what trade-offs exist between latency and video quality.

Together these two approaches reflect the dual nature of the thesis: it is both a development project and a measurement study, and the methodology is designed to support both goals.

4.5 Development

4.5.1 GStreamer Pipeline Development

4.6 Experimental Methodology

4.6.1 Latency Measurement

5 Results

6 Conclusion

6.1 Evaluation of latency

References

- [1] Simon Jürgensen. ‘Remote Control for Rail Vehicles’. PhD thesis. University of Kiel, 2025. URL: <https://rtsys.informatik.uni-kiel.de/~biblio/downloads/theses/sij-bt.pdf> (visited on 15th Oct. 2025).
- [2] Inzirillo Francesco et al. *D5.4 - Documentation of use cases for Remote Driving*. FP2R2DATO and Europe’s Rail, 2023. URL: <https://rail-research.europa.eu/wp-content/uploads/2025/07/D5.4-%E2%80%93Documentation-of-use-cases-for-Remote-Driving.pdf> (visited on 8th Dec. 2025).
- [3] International Electrotechnical Commission (IEC). *IEC 62290-1:2025 -Railway applications - Urban guided transport management and command/control systems - Part 3: System requirements specification*. IEC, 2025. URL: <https://online.standard.no/nb/nek-iec-62290-3-2025>.
- [4] International Association of Public Transport (UITP). *World Report on Metro Automation*. UITP, 2018. URL: <https://www.uitp.org/publications/world-report-on-metro-automation/> (visited on 15th Oct. 2025).
- [5] Europe’s Rail FP6 WP3. *D3.1 – Use Cases and Scenarios for ATO in Different GoA Demos for G1 Regional Lines*. Tech. rep. Europe’s Rail, 2025. URL: <https://rail-research.europa.eu/rail-projects/outputs/d3-1-uc-and-scenarios-for-ato-in-different-go-a-demos-for-g1-regional-lines/> (visited on 20th Mar. 2026).
- [6] Office of Rail and Road. *Railway Safety Principles and Factors for Safe Operation of GoA4 UTO*. Tech. rep. Office of Rail and Road, 2024. URL: <https://www.orr.gov.uk/sites/default/files/2024-07/railway-safety-principles-factors-for-safe-operation-of-go-a4-uto.pdf> (visited on 20th Mar. 2026).
- [7] Niels Brandenburger, Friedrich Maximilian Strauß and Anja Naumann. *Effects of Video Quality on Perception and Reaction Time in Video-based Remote Train Control*. DLR, 2023. URL: <https://elib.dlr.de/194929/1/PresentationHFES2023.DLR.pdf> (visited on 5th Dec. 2025).
- [8] Emilia Salvesen Kozarevic. ‘Developing Remote Control as a Fallback System: An Iterative Approach’. PhD thesis. NTNU, 2025.
- [9] Einar Thingstad Myhrvold. ‘Latency for Remote Control in ATO a Scoping Review’. PhD thesis. NTNU, 2025.
- [10] ETSI. *ETSI TS 122 289 V17.0.0 LTE 5G Mobile communication system for railways 3GPP TS 22.289 version 17.0.0 Release 17*. Tech. rep. 2022. URL: https://www.etsi.org/deliver/etsi_ts/122200_122299/122289/17.00.00_60/ts-122289v170000p.pdf.
- [11] Nacho Celaya Vela et al. *D41.2 - Testing reports and assessment*. CAF et al., 2024. URL: <https://rail-research.europa.eu/wp-content/uploads/2025/02/D41.2-Testing-reports-assessment.pdf> (visited on 7th Dec. 2025).
- [12] Stefan Neumeier et al. ‘Teleoperation: The Holy Grail to Solve Problems of Automated Driving? Sure, but Latency Matters’. In: (2019). URL: <https://dl.acm.org/doi/pdf/10.1145/3342197.3344534> (visited on 15th Oct. 2025).
- [13] Iraj Sodagar. ‘The MPEG-DASH Standard for Multimedia Streaming Over the Internet’. In: *IEEE MultiMedia* 18.4 (2011), pp. 62–67. URL: <https://ieeexplore.ieee.org/document/5751395> (visited on 20th May 2026).
- [14] Daniel Mejías et al. *Towards Railways Remote Driving: Analysis of Video Streaming Latency and Adaptive Rate Control*. 2024. URL: <https://ieeexplore.ieee.org/document/10597107> (visited on 6th Dec. 2025).
- [15] Thomas Wiegand et al. ‘Overview of the H.264/AVC Video Coding Standard’. In: *IEEE Transactions on Circuits and Systems for Video Technology* 13.7 (2003), pp. 560–576. URL: <https://ieeexplore.ieee.org/document/1218189> (visited on 20th May 2026).
- [16] Gary J. Sullivan, Pankaj Topiwala and Ajay Luthra. ‘The H.264/AVC Advanced Video Coding Standard: Overview and Introduction to the Fidelity Range Extensions’. In: *Proceedings of SPIE* 5558 (2004). URL: <https://www.semanticscholar.org/paper/27c3343314940d32536158095113f5711e9d47c8> (visited on 20th May 2026).

-
- [17] Gary J. Sullivan et al. ‘Overview of the High Efficiency Video Coding (HEVC) Standard’. In: *IEEE Transactions on Circuits and Systems for Video Technology* 22.12 (2012), pp. 1649–1668. URL: <https://ieeexplore.ieee.org/document/6316136> (visited on 20th May 2026).
 - [18] Jens-Rainer Ohm et al. ‘Comparison of the Coding Efficiency of Video Coding Standards Including High Efficiency Video Coding (HEVC)’. In: *IEEE Transactions on Circuits and Systems for Video Technology* 22.12 (2012), pp. 1669–1684. URL: <https://ieeexplore.ieee.org/document/6317158> (visited on 20th May 2026).
 - [19] Wikipedia contributors. *Motion JPEG*. 2026. URL: https://en.wikipedia.org/wiki/Motion_JPEG (visited on 20th May 2026).
 - [20] e-con Systems. *A Comprehensive Study: MJPEG and H.264 Compression in Embedded Vision*. 2024. URL: <https://www.e-consystems.com/blog/camera/technology/a-comprehensive-study-mjpeg-and-h-264-compression-in-embedded-vision/> (visited on 20th May 2026).
 - [21] Admir Kaknjo et al. ‘Real-Time Video Latency Measurement between a Robot and Its Remote Control Station: Causes and Mitigation’. In: *Wireless Communications and Mobile Computing* (2018). URL: https://www.researchgate.net/publication/329369713_Real-Time_Video_Latency_Measurement_between_a_Robot_and_Its_Remote_Control_Station_Causes_and_Mitigation (visited on 15th Oct. 2025).